C PROGRAMMING — ASSIGNMENT 1

TASK 2: CHOOSE THE DATA TYPES

CO1 • Bloom's Level: K4 – Analyse • Marks: 4

Assignment title: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Purpose: Select suitable C data types, examine their sizes and ranges, handle a 16-bit meter rollover, and explain why integer paise is safer than floating-point rupees for billing.

1. Data-type selection and justification

The program processes meter readings, household identifiers, unit counts and bill amounts. The exact sizes shown by `sizeof` depend on the compiler and platform. The uploaded execution screenshot reports the sizes and limits below; these values should be treated as the observed result for that environment.

Data / purpose	Suggested C type	Observed size / range	Reason
Meter register	`unsigned short` (typedef `meter_t`)	2 bytes; 0–65535	Matches the stated 16-bit unsigned meter counter.
Consumption units	`unsigned int`	4 bytes; 0–4,294,967,295 on a 32-bit unsigned int	Non-negative unit differences, including rollover.
Household number / ordinary count	`int`	4 bytes; screenshot INT_MAX = 2,147,483,647	Suitable for normal identifiers and loop counts within range.
Large accumulated totals	`long long` or `unsigned long long`	8 bytes in screenshot; LLONG_MAX = 9,223,372,036,854,775,807	Provides headroom for district-scale totals.
Money	`long long` paise	Integer; exact to 1 paise	Avoids binary floating-point rounding in financial totals.
Fractional scientific values	`double`	8 bytes; DBL_DIG = 15 in screenshot	Use when fractional values are required, not as the primary money store.

The screenshot also reports: `char` = 1 byte, `short` = 2 bytes, `int` = 4 bytes, `long` = 8 bytes, `long long` = 8 bytes, `float` = 4 bytes, and `double` = 8 bytes. These are platform observations, not universal sizes guaranteed on every C implementation.

2. Why store money as integer paise?

A rupee amount can contain two decimal places. Storing rupees in `float` uses binary floating-point, which cannot represent many decimal fractions exactly. Small errors can accumulate when thousands of bills are added. Storing the amount as integer paise (for example, ■162.50 as 16250 paise) keeps each addition exact, provided the chosen integer type does not overflow. Convert to rupees and paise only when displaying the result.

Illustrative accumulation: if 4,000 bills are each ■0.10, the exact total is ■400.00. Repeated floating-point addition may produce a value slightly above or below 400.00 depending on representation and evaluation; integer paise gives 1,000 paise × 4,000 = 400,000 paise = ■4,000.00 for ■1.00 bills, or 10 paise × 4,000 = 40,000 paise = ■400.00 for ■0.10 bills.

3. Completed code — meter rollover

```
#include <stdio.h>
#include <limits.h>
```

```c
/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    /* Unsigned 16-bit subtraction wraps modulo 65536. */
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */
    return 0;
}
```

4. Execution evidence supplied

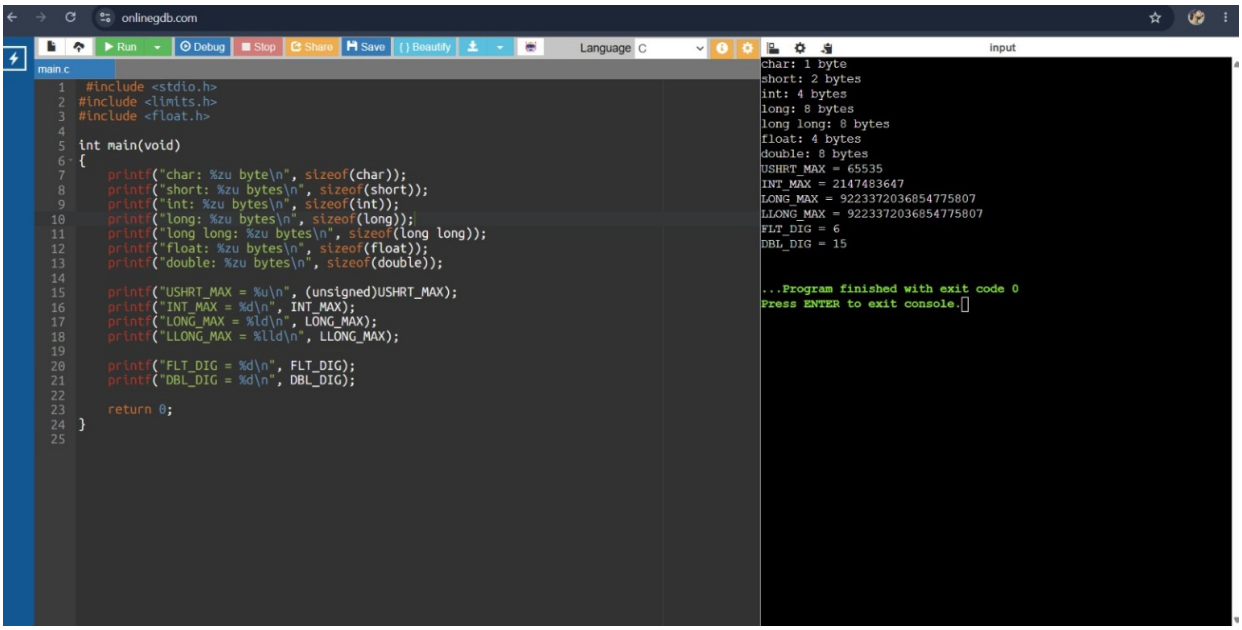


Figure 1. Screenshot supplied for Task 2: observed type sizes, integer limits and floating-point digit limits.

The screenshot shows the following output values:

Item	Observed output
char / short / int / long / long long	1 / 2 / 4 / 8 / 8 bytes
float / double	4 / 8 bytes
USHRT_MAX	65535
INT_MAX	2147483647
LONG_MAX / LLONG_MAX	9223372036854775807
FLT_DIG / DBL_DIG	6 / 15

5. Expected output for the completed rollover program

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Explanation of rollover result

An unsigned 16-bit register has 65,536 possible values, from 0 through 65,535. When the previous reading is 65,400 and the current reading is 120 after rollover, the consumed units are (65,536 – 65,400) + 120 = 136 + 120 = 256.

Unsigned arithmetic wraps modulo 65,536, so the result is a non-negative count rather than a negative number. The code assumes at most one rollover occurred between readings.

6. Alternative concept: double rupees vs integer paise

With `double`, the program would store a bill as a floating-point rupee value and round to two decimal places when the bill is finalized or displayed. Rounding only at the end can allow small representation errors to accumulate; rounding each bill to paise before summing keeps the billing rule consistent. Both approaches take O(1) time per addition. Each value also uses O(1) space, but a double occupies 8 bytes on the observed system while a 64-bit integer paise value also occupies 8 bytes and represents paise exactly within its range. For n bills, floating-point error can accumulate with n operations; integer-paise addition introduces no rounding error, though integer overflow must still be checked.

7. Compile and run

Save the program as `task02.c` and compile with GCC:

```
gcc -std=c11 -Wall -Wextra task02.c -o task02
./task02
```

The source code and screenshot should be saved inside the repository folder `Task02`. Add this PDF, the C source file, and the output screenshot to that folder. The assignment requires a progressive commit for each task and a README.md with the compiler version, exact command, run steps, and a short result note.

Conclusion

For reliable electricity billing, use unsigned integer types for non-negative meter values, wider integer types for large totals, and integer paise for money. The 16-bit rollover calculation gives 256 units for the example in the assignment. Always verify type sizes and limits on the actual compiler, and check that accumulated totals remain within the selected type’s range.